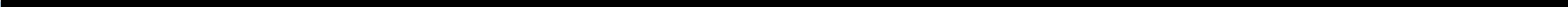




■ **Compiladores**

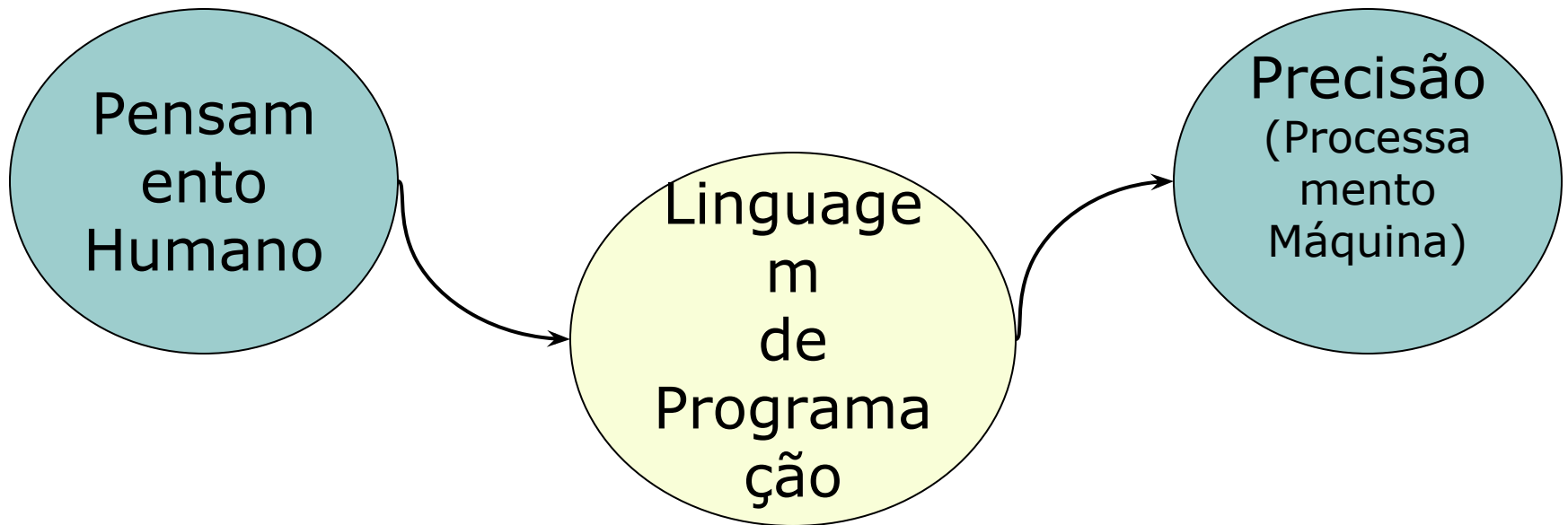
- Introdução -



■ Sumário

- Introdução
- Tradutores
- Estrutura de um Tradutor
- Geradores de Compiladores

■ Introdução



- 1ª Geração – Linguagem de Máquina (bits, registradores...)
- 2ª Geração – Linguagens Simbólicas (Assembly)
- 3ª Geração – Linguagens Voltadas ao Usuário (Pascal, C, Ada)
- 4ª Geração – Linguagens Orientadas à Aplicação
- 5ª Geração – Linguagens de Conhecimento (PROLOG)

■ Tradutor

- É um programa que aceita um programa escrito em uma linguagem de programação (**fonte**) e produz como resultado um programa equivalente em outra linguagem (**objeto**).
- Montadores
- Macro-assemblers
- Compiladores
- Pré-compiladores, pré-processadores ou filtros
- Interpretadores

■ Montadores ou Assemblers

- Usados para traduzir **linguagem simbólica** (Assembly) para instruções de máquina
- Relação de uma para uma
 - Uma instrução em linguagem simbólica para uma instrução de máquina

■ Macro-assemblers

- Usados para traduzir **linguagem simbólica** para instruções de máquina
 - Linguagens simbólicas com possibilidade de definição de macros
- Relação uma para várias
 - Um comando macro é traduzido para uma sequência de instruções simbólicas antes de ser traduzido para linguagem de máquina

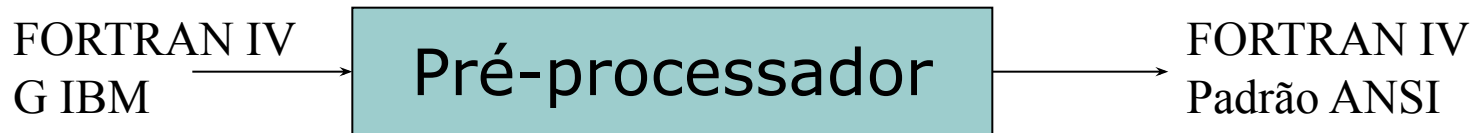
Compiladores e Pré-compiladores

■ Compiladores

- Traduzem **linguagem de alto nível** para programas equivalentes em **linguagem simbólica** ou **linguagem de máquina**

■ Pré-compiladores, pré-processadores ou filtros

- Efetuam traduções entre duas linguagens de alto nível



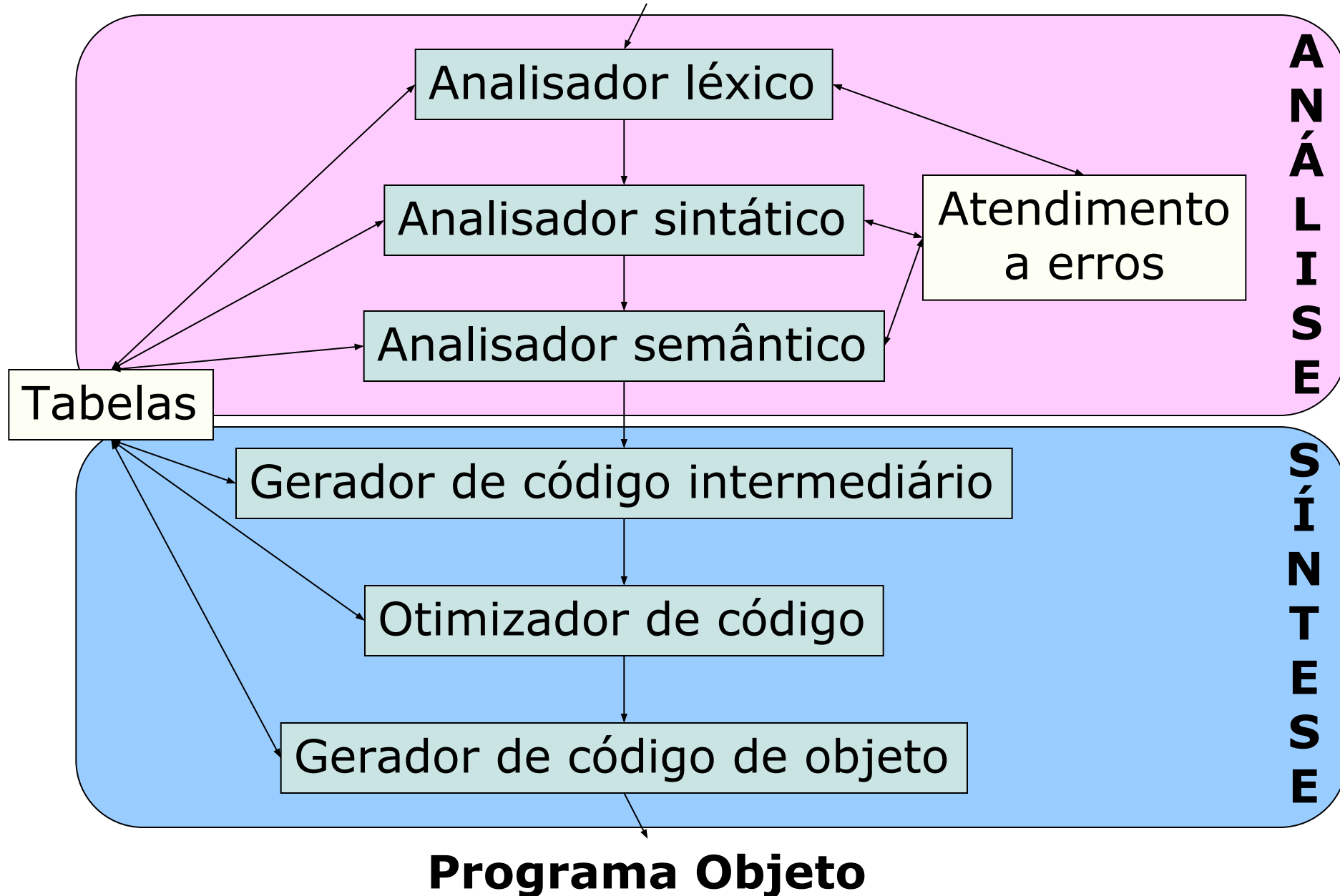
■ Interpretadores

- Recebem o código intermediário de um programa anteriormente traduzido e produzem o “efeito de execução” do algoritmo original – sem mapeá-lo para linguagem de máquina
- Não é gerado programa objeto
- A interpretação ocorre em tempo de execução

■ Estrutura de um tradutor

- Tradutores são programas complexos
- Possuem estrutura semelhante
 - Funções padronizadas para análise do programa fonte e posterior síntese para a derivação do código objeto;
- São estruturados em fases
 - cada fase se comunica com a seguinte através de uma linguagem intermediária;
 - na prática, podem não estar individualizadas

Programa Fonte



■ Análise

Envolve tarefas analíticas:

- Léxico
- Sintático
- Semântico

■ Análise Léxica

- Denominado ***scanner***
- Função
 - *"Fazer a leitura do programa fonte, caractere a caractere, e traduzí-lo para uma sequência de símbolos léxicos, também chamados **tokens**"*
- **Símbolos léxicos:** palavras reservadas, identificadores, constantes e operadores da linguagem

■ Análise Léxica (2)

- Verifica se os caracteres lidos do programa fonte pertencem ao alfabeto da linguagem, identificando ***tokens*** e desprezando comentários e brancos desnecessários
- Inicia a construção da tabela de símbolos
- Envia mensagens de erro, se identificar unidades léxicas não aceitas pela linguagem

■ Análise Léxica (3)

- É implementado como uma subrotina sob o comando do Analisador Sintático
- **Saída:** cadeia de tokens para a Análise Sintática
- **Token:** pode ser representado
 - Pelo próprio símbolo
 - ou
 - Por um par ordenado:
[classe do símbolo, índice para a tabela]

■ Análise Léxica - exemplo

```
while I < 100 do I := J + I
```

- Os *tokens* gerados seriam:

```
[while, ] [id, 7] [<, ] [cte, 13] [do, ]  
[id, 7] [:=, ] [id, 12] [+, ] [id, 7]
```

- As palavras reservadas e operadores são representados pelos próprios símbolos
- Identificadores de variáveis e constantes numéricas pelo par ordenado com o índice da tabela de símbolos

■ Análise Sintática

- Função
 - *"Verificar se a estrutura gramatical do programa foi formada usando as regras gramaticais da linguagem"*
- Identifica sequências de símbolos que constituem **estruturas sintáticas** (expressões, comandos)
- Faz uma varredura (*parsing*) nos tokens
- Produz uma **Árvore de Derivação** que exhibe a estrutura sintática do texto fonte
- Também: detecta erros de sintaxe (identifica a posição e o tipo de erro)

■ Análise Semântica

- Função
 - *"Verificar se as estruturas do programa irão fazer sentido durante a execução"*
- Por exemplo:
 - verificar se um identificador declarado como variável está sendo usado como tal
 - verificar compatibilidade entre os tipos (operandos, operadores)

Analizador Sintático e Semântico

- Geralmente o Analizador Sintático trabalha conjuntamente com o Analizador Semântico
- **Exemplo** de sintaxe (<expressão> deve estar sintaticamente correta e sua avaliação deve retornar um valor do tipo lógico) :

While <expressão> do <comando>

- As regras gramaticais que definem as construções da linguagem podem ser descritas através de produções cujos elementos incluem *símbolos terminais e não-terminais*

Analizador Sintático e Semântico - exemplo

- No exemplo anterior, os comandos while e de atribuição podem ser definidos pela gramática $G = (V, T, P, S)$, onde:
 - T são os símbolos em negrito e azul (terminais)
 - $T = \{\textbf{while}, \textbf{do}, \textbf{:=}, \textbf{<}, \textbf{+}, \textbf{I}, \textbf{J}, \textbf{100}}\}$
 - V são os símbolos não terminais (variáveis) definidos entre os sinais < e >
 - $V = \{\langle \text{comando} \rangle, \langle \text{while} \rangle, \langle \text{atrib} \rangle, \langle \text{expr_bool} \rangle, \langle \text{variavel} \rangle, \langle \text{expr_arit} \rangle, \langle \text{expr_bool} \rangle, \langle \text{termo} \rangle, \langle \text{numero} \rangle\}$
 - $S = \langle \text{comando} \rangle$
 - $P = \dots$

Analizador Sintático e Semântico - exemplo

- $P = \{$
 - $\langle \text{comando} \rangle \rightarrow \langle \text{while} \rangle \mid \langle \text{atrib} \rangle \mid \dots$
 - $\langle \text{while} \rangle \rightarrow \text{while} \langle \text{expr_bool} \rangle \text{do} \langle \text{comando} \rangle$
 - $\langle \text{atrib} \rangle \rightarrow \langle \text{variavel} \rangle := \langle \text{expr_arit} \rangle$
 - $\langle \text{expr_bool} \rangle \rightarrow \langle \text{expr_arit} \rangle < \langle \text{expr_arit} \rangle$
 - $\langle \text{expr_arit} \rangle \rightarrow \langle \text{expr_arit} \rangle + \langle \text{termo} \rangle \mid \langle \text{termo} \rangle$
 - $\langle \text{termo} \rangle \rightarrow \langle \text{numero} \rangle \mid \langle \text{variavel} \rangle$
 - $\langle \text{variavel} \rangle \rightarrow \mathbf{I} \mid \mathbf{J}$
 - $\langle \text{numero} \rangle \rightarrow \mathbf{100}$
- }

Qual a árvore de derivação para:

while I < 100 do I := J + I ?

■ Considerações – Análise

- Para o **Analizador Léxico**, o programa fonte é uma sequência de palavras de uma **linguagem regular**
- É especializado no reconhecimento do texto fonte original, fazendo a leitura dos caracteres e gerando os ***tokens***

■ Considerações – Análise (2)

- Para o **Analizador Sintático**, a sequência de *tokens* gerada pela análise léxica é uma sequência de palavras de uma **linguagem livre do contexto**.
 - Essa sequência deve satisfazer às regras gramaticais de uma LLC
 - Os *tokens* são os símbolos terminais da GLC

■ Síntese

Construção do código objeto

- Código intermediário
- Otimização
- Código objeto

Geração de Código Intermediário

- Gera uma sequência de código a partir da saída do Analisador Sintático
- É um código intermediário
 - Não especifica detalhes como: registradores usados, endereços de memória...
- Tradução em mais de um passo
 - Possibilita otimização do código
 - Resolve gradualmente a tradução
- Nos tradutores simples, esta fase não existe

Geração de Código Intermediário - exemplo

- A partir da árvore de derivação anterior, o código intermediário gerado é:

```
L0  if I < 100 goto L1
    goto L2
L1  TEMP := J + I
    I := TEMP
    goto L0
L2  ...
```

- Os códigos intermediários podem ser em notação polonesa pós-fixada, quádruplas, triplas..

■ Otimização de Código

- Tornar o código eficiente
 - Velocidade de execução
 - Utilização memória

■ Exemplo

```
L0  if I >= 100 goto L2
    I := J + I
    goto L0
L2  ...
```

Geração de Código Objeto

- Geração do código de máquina
 - Gera o código de máquina fazendo a reserva de memória para constantes e variáveis, e seleciona registradores
- Fase mais complexa
- Exemplo (linguagem simbólica)

```
L0 MOV AX, I
    CMP AX, 100
    JGE L2
    MOV AX, J
    MOV BX, I
    ADD BX, AX
    MOV I, BX
    JMP L0
```

```
L2 ...
```



Outros componentes da estrutura

Gerência de Tabelas
Atendimento a Erros

■ Gerência de Tabelas

- Conjunto de tabelas e rotinas utilizadas por todas as fases do compilador
 - Tabelas de palavras reservadas
 - Tabelas de delimitadores
 - **Tabela de Símbolos** (montada durante a análise):
 - Declaração de variáveis: classe (var), tipo, endereço no texto, precisão, tamanho
 - Declarações dos procedimentos e rotinas: classe (proc), número de parâmetros
 - Parâmetros de subrotinas: classe (par), tipo, mecanismo de passagem

■ Atendimento a erros

- Tratamento de erros
 - Fase analítica
 - prossegue a análise, mesmo que erros tenham sido detectados



Geradores de Compiladores

Analisadores Léxicos
Analisadores Sintáticos
Geradores de Código

Sistemas geradores de compiladores

- Geradores de Analisadores Léxicos
 - Geram automaticamente reconhecedores para os símbolos léxicos (palavras chaves, identificadores, operadores...)
 - Esses reconhecedores baseiam-se em **gramáticas regulares** ou **expressões regulares**

Sistemas geradores de compiladores (2)

- Geradores de Analisadores Sintáticos
 - Produzem reconhecedores sintáticos a partir de **gramáticas livres do contexto**
- Geradores de Geradores de Código
 - Recebem regras que definem a tradução de cada operação da linguagem e fazem o mapeamento para esqueletos que representam sequências de instruções de máquina

■ Bibliografia

- AHO, Alfred V.; SETHI, Ravi; ULLMAN, Jeffrey D. 1995. **Compiladores Princípios, Técnicas e Ferramentas**. Rio de Janeiro: LTC
- PRICE, A M. A.; TOSCANI, S. S. 2001. **Implementação de Linguagens de Programação**. Porto Alegre: Sagra Luzzatto.